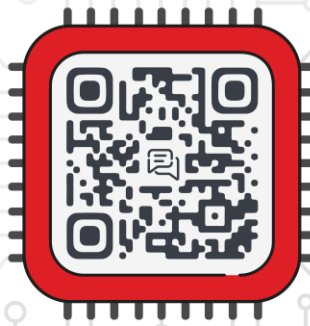




نظري

1

30

3600 sp

كلية الهندسة المعلوماتية

السنة الرابعة - النظم والشبكات

Introduction and Tcp Sockets

د. غسان شددود



RB Informatics; محتوى مجاني غير مخصص للبيع التجاري
5/11/2025

برمجة تطبيقات شبكية

أهلاً وسهلاً بكم في المحاضرة الأولى من مادة اختصاصنا في هذا الفصل ، سنتناول مادة التطبيقات الشبكية عدة مفاهيم ، أهمها :

- مفهوم الشبكات وتطبيقها

- port number , IP

- بروتوكولات ال UDP. Tcp

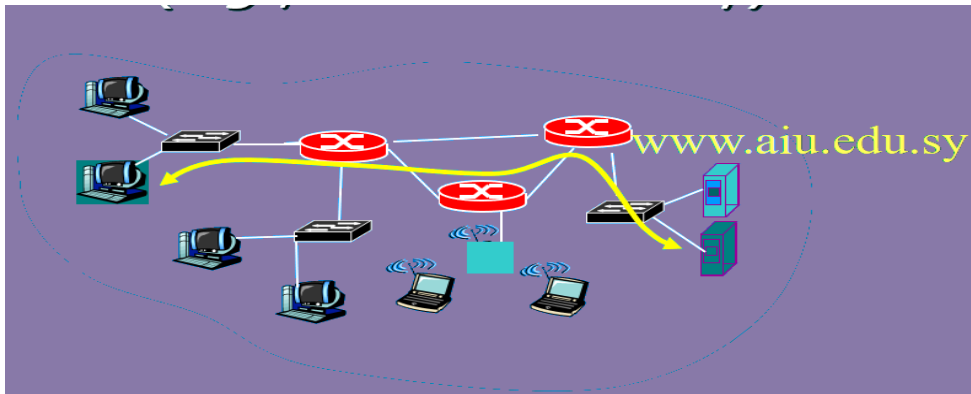
- بروتوكولات ال IPv4,IPv6

- أنظمة الاتصال وأنواعها

- مفاهيم ال: Design pattern ,Threads ,processes

لنبدأ :

الشبكة هي مجموعة أجهزة قادرة على التواصل مع بعضها سلكياً أو لاسلكياً و قادرة على تبادل المعلومات فيما بينها



router



Switch



PC



mobile



server

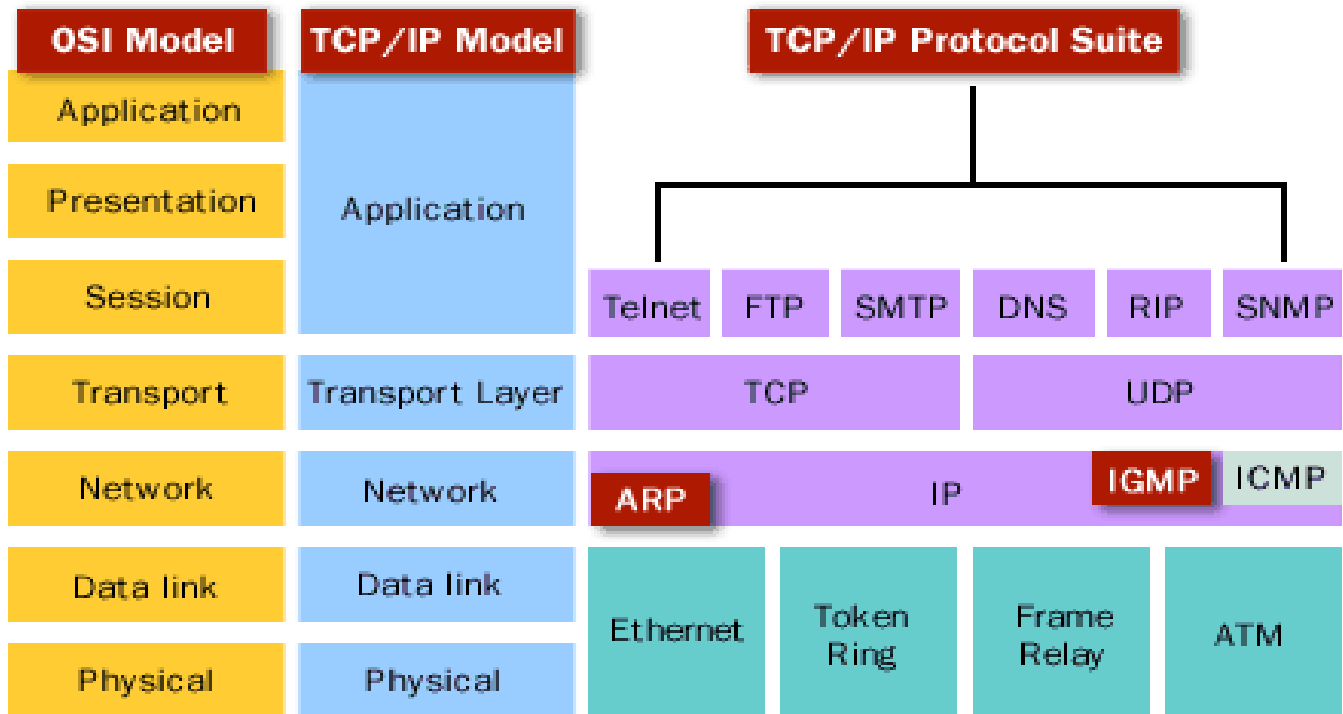
host

الصورة أعلاه تظهر شبكة حاسوبية ، مكونة من مجموعة مضيفين (PCs, mobiles,server) موزعة على أربع شبكات و Lan متصلة مع بعضها من خلال routers ,switches ويوجد نقطة نفاذ (access point) تسمح بنفاذ الأجهزة إلى الشبكة لاسلكياً.

هذه الشبكة تسمح بتبادل المعلومات فيما بين الأجهزة الموجودة داخلها ، كل مضيف لديه عنوان خاص به (host name) او (host Address) كما في الصورة .

Networks

وهي سلسلة من الطبقات Layers تدعى stack ، مثل Tcp/IP (كما هو موضح بالصورة) :



Internet protocol : هو بروتوكول مسؤول عن نقل الداتا end to end بغض النظر عن الانظمة الوسيطة التي يمر خلالها .

تتبادل هذه الطبقات البيانات فيما بينها باستخدام البروتوكولات، حيث يحدد البروتوكول القواعد التي يجب على كل من المرسل والمستقبل وجميع الأجهزة الوسيطة اتباعها ليتمكنوا من التواصل بفعالية. كل طبقة من الطبقات توفر خدمة للطبقة التي تعلوها.

DNS : Domain name system

NFS : Network file system

Internet Protocols

IP is the main protocol in the internet layer

و هنالك نسختان منه :

1. 32 bits طوله IPv4

2. 128 bits طوله IPv6

توفر خدمات ال IP خدمات توفير لاسلكية ، ويحمل داخله بيانات طبقة النقل .
هناك بروتوكولين في طبقة الانترنت أيضاً :

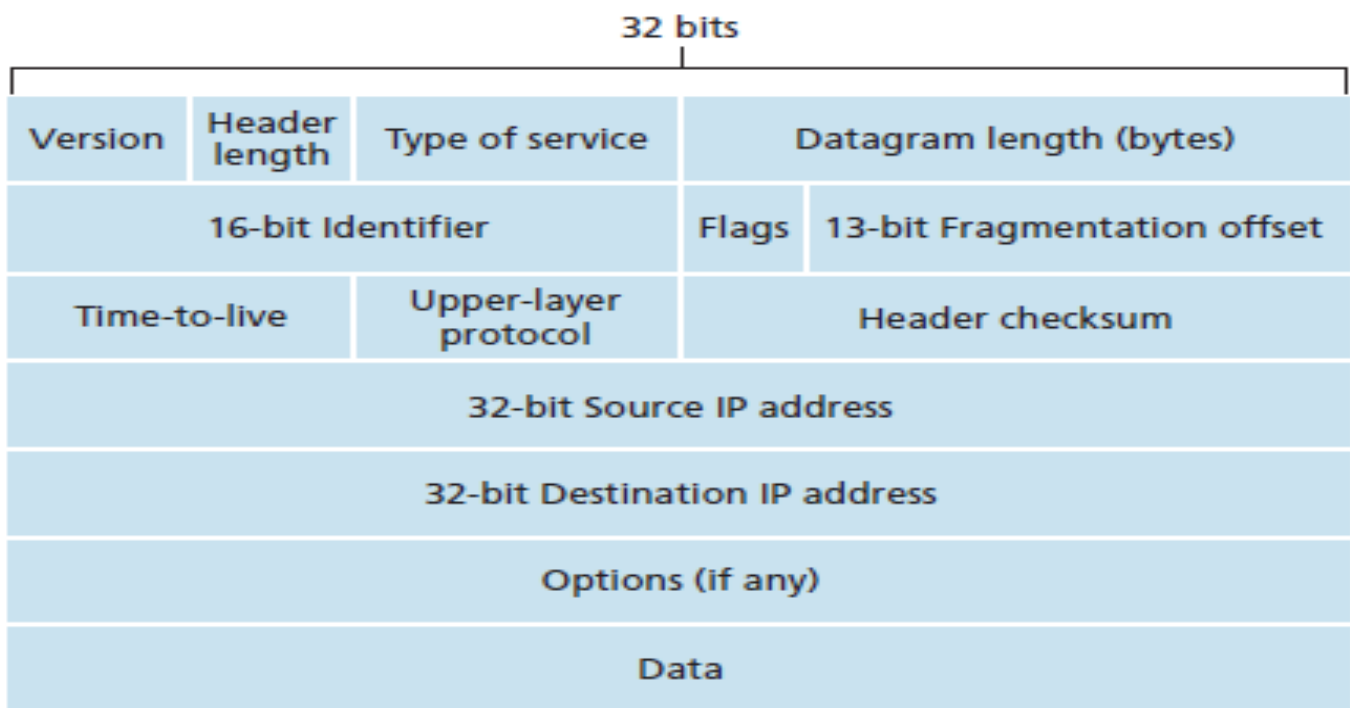
ICMP : تزود بخدمات التحكم والانذار والتي تعتمد برامج Ping , Trace route
IGMP : تستخدم في الاتصالات الجماعية (multi casting)

IPv4 Datagram:

كل حزمة بيانات IPv4 تحتوي على Header حجمها بين 60 → 0 byte بيانات يصل حجمها الى 65535 byte

وفيه قسمين مهمين هما :

- source address (bytes 4)
 - destination address (bytes 4)
- و يكون شكل توزيع المرسلات فيه كالتالي :



IP Address and Domain Names

كل جهاز حاسوب متصل بشبكة IPv4 عنوان وحيد مكون من 4 بايتات يتضمن ال Header عند نقل البيانات عنوان ومصدر الوجهة.

تم تطوير نظام يدعى DNS (Domain name system) وذلك لترجمة الاسماء المتعلقة بكل جهاز الى IP خاص به.
مُثلاً :

host name : www.aiu.edu.sy

يترجمه الى :

IP address : 213.178.247.220

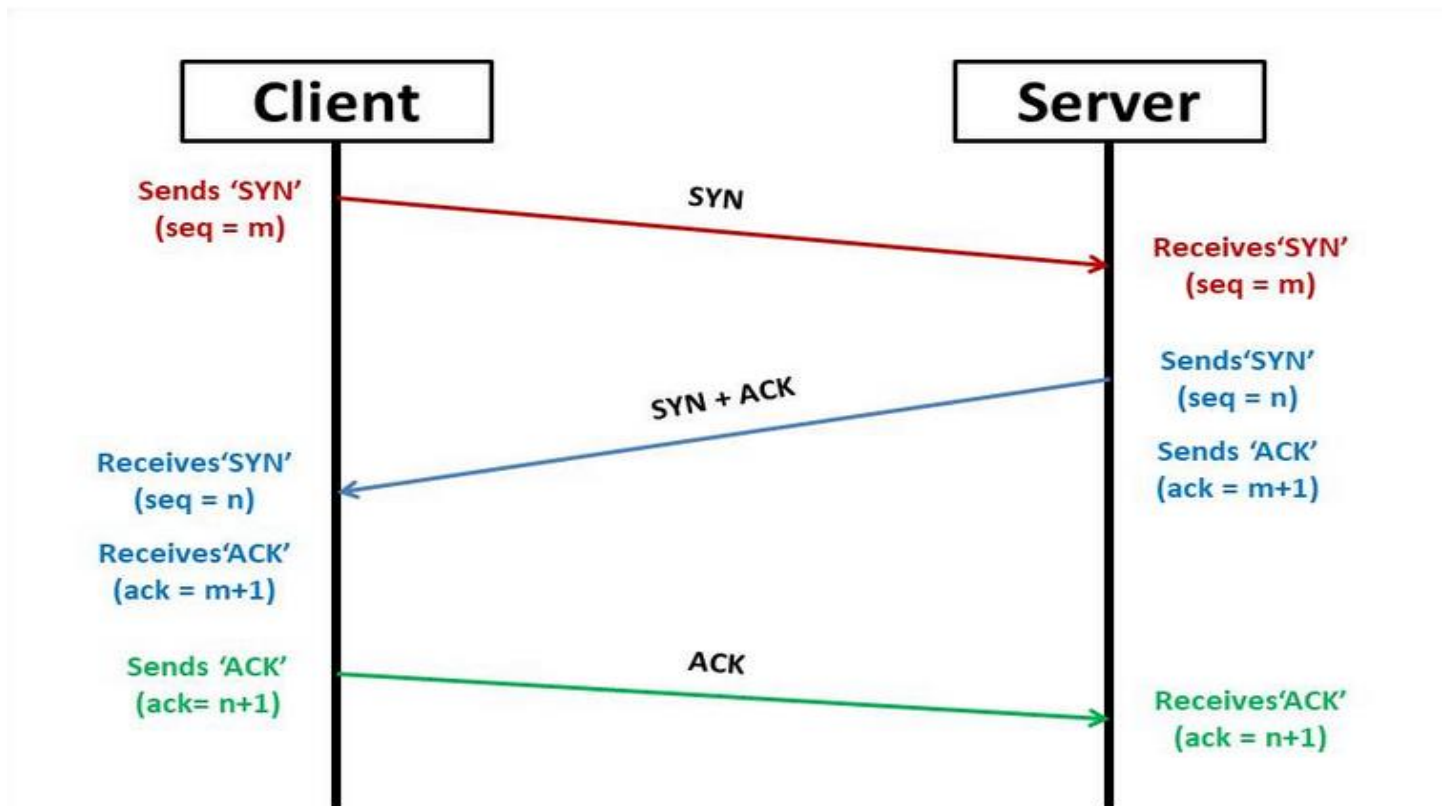
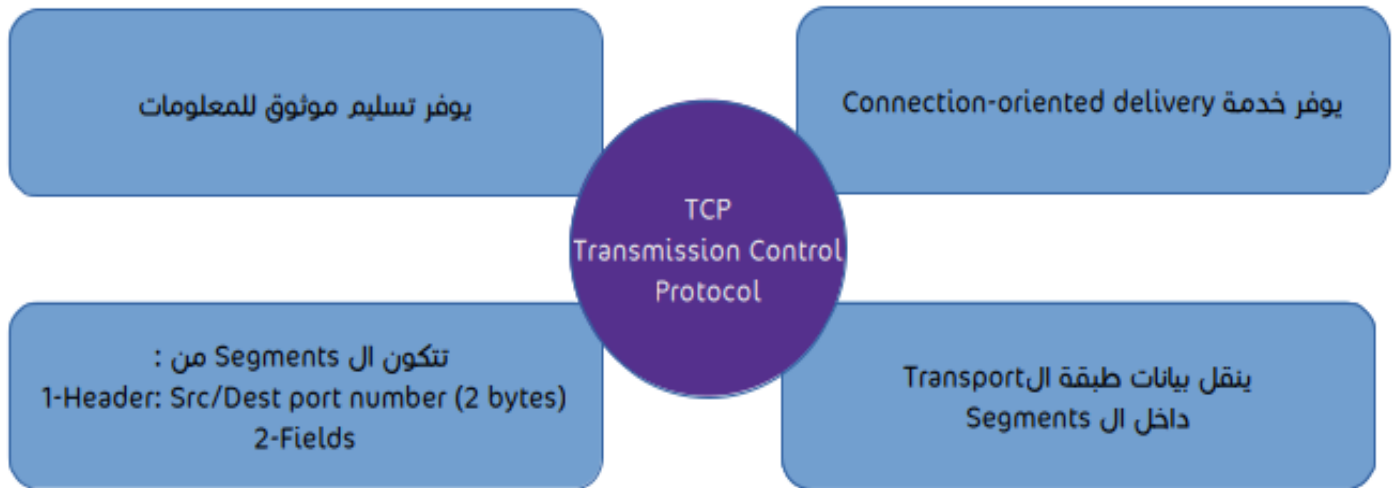
يتم تقسيم المخطط إلى مكونين رئيسيين:
مكدس TCP/IP اليمين: وهو هيكل الطبقات الذي ينظم عمل الشبكة.
حزمة بروتوكولات TCP/IP اليسار: وهي القواعد والإجراءات الفعلية التي تعمل داخل كل طبقة.

الطبقة (Layer)	الوظيفة الأساسية	أمثلة على البروتوكولات
1. التطبيق (Application)	توفير واجهة للتطبيقات التي يستخدمها المستخدم.	HTTP (الويب)، FTP (نقل الملفات)، SMTP (البريد)، DNS (ترجمة الأسماء).
2. النقل (Transport)	إدارة الاتصال بين جهازين مضيفين (End-to-End).	TCP (توصيل موثوق)، UDP (توصيل سريع وغير موثوق).
3. الإنترنت (Internet)	عنونة البيانات وتوجيهها (Routing) عبر شبكات مختلفة.	IP (بروتوكول الإنترنت)، ARP (ترجمة العنوان).
4. المضيف إلى الشبكة (Host-to-Network)	التعامل مع التفاصيل المادية والمنطقية للإرسال عبر الوسيط (الكابل، الواي فاي).	PPP, Wi-Fi, Ethernet.

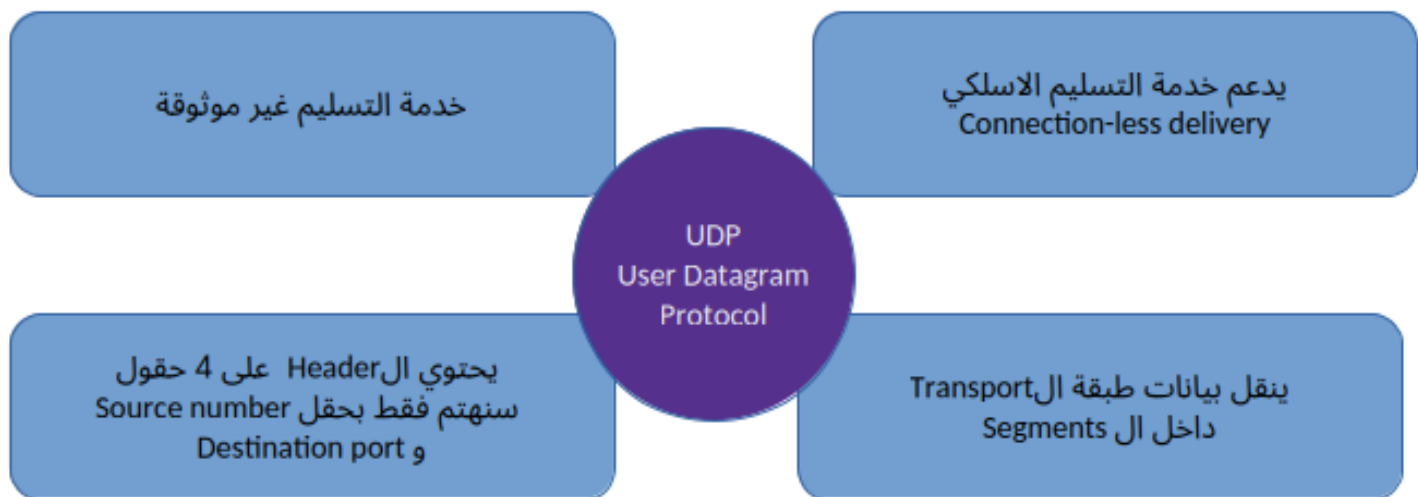
بروتوكول HTTP (HyperText Transfer Protocol) وظيفته الرئيسية هي (نقل صفحات الويب وملفاتهما) مثل نصوص HTML والصور بين متصفحك (العميل) وخادم الويب).

مثال عملي: النقل والإنترنت
طبقة النقل (TCP/UDP) تُنشئ قطعة بيانات (Segment) لضمان موثوقيتها أو سرعتها.
تطلب طبقة النقل خدمة من طبقة الإنترنت (IP)، وتقول: "يا IP، خذ قطعة البيانات هذه وقم بتسليمها إلى هذا العنوان الرقمي (IP Address)".
بروتوكول IP لا يهتم بما بداخل قطعة البيانات سواء كانت HTTP أو FTP؛ مهمته الوحيدة هي توجيهها عبر الشبكات المختلفة للوصول إلى الوجهة النهائية.

Transport Layer Protocols



هذه الصورة تشرح عملية (three way handshaking) في بروتوكول TCP بين العميل والخادم، حيث يتبادلون رسائل SYN وACK لإنشاء اتصال آمن بين الطرفين.



The Application Layer

هي الطبقة التي تقوم بتسليم : البيانات إلى المستخدم ، وتحدد بروتوكول هذه الطبقة الذي سيستخدم من أجل الخدمة.

بروتوكولات طبقة ال Application :

-HTTP: Hyper Text Transfer Protocol

مثلا يقوم بالتأكد ان الصورة تعرض صحيحة وليس تسلسل ارقام طويل

-SMTP: Simple Mail Transfer Protocol

-POP: Post Office Protocols

-IMAP: Internet Message Access Protocol

3 بروتوكولات خاصة بالبريد الالكتروني (Email)

-FTP: File Transfer Protocol

-TFTP: Trivial File Transfer Protocol

بروتوكولان يستخدمان لنقل الملفات

-DNS: Domain Name Service

بروتوكول يعيد IP المضيف الذي يصله اسمه

Ports

- ال addresses تعرف hosts , ال ports تعرف programs .
- كل بورت يتم تعريفه برقم يتراوح بين 1 → 65535 .
- عند انشاء نقطة اتصال يجب انشاء socket والتي تعرف ب IP و port number , حيث يتم الاتصال بأحد بروتوكولات الاتصال (UDP, TCP) .
- ارقام البورتات المترواحة بين 1 → 1024 هي محجوزة لخدمات معينة مثل :
 - 80: معرف لخدمة الويب HTTP
 - 23: معرف لخدمة ال Telnet
 - 25: معرف ل email sender

Network Applications

هي عبارة عن برامج تستخدم الانترنت أو اي بنية شبكية أخرى لتنفيذ وظائف مفيدة ومحددة، مثل نقل الملفات داخل الشبكة ، تصفح مواقع الويب، المراسلة الفورية و غيرها من الخدمات التي تعتمد على الاتصال الشبكي .

أما : طبقة ال Application لا تشير إلى البرامج أو التطبيقات البرمجية ، وإنما هي طبقة وظيفية (interface) تسمح لهذه البرامج بالوصول إلى موارد الشبكة والتواصل مع باقي الأنظمة عبرها .

و يمكن فيها تعريف بروتوكولات تتناسب مع متطلبات الخدمة ، بحيث يحدد البروتوكول طريقة تبادل البيانات والتواصل بين الأنظمة في الطرفين .



Interaction Models

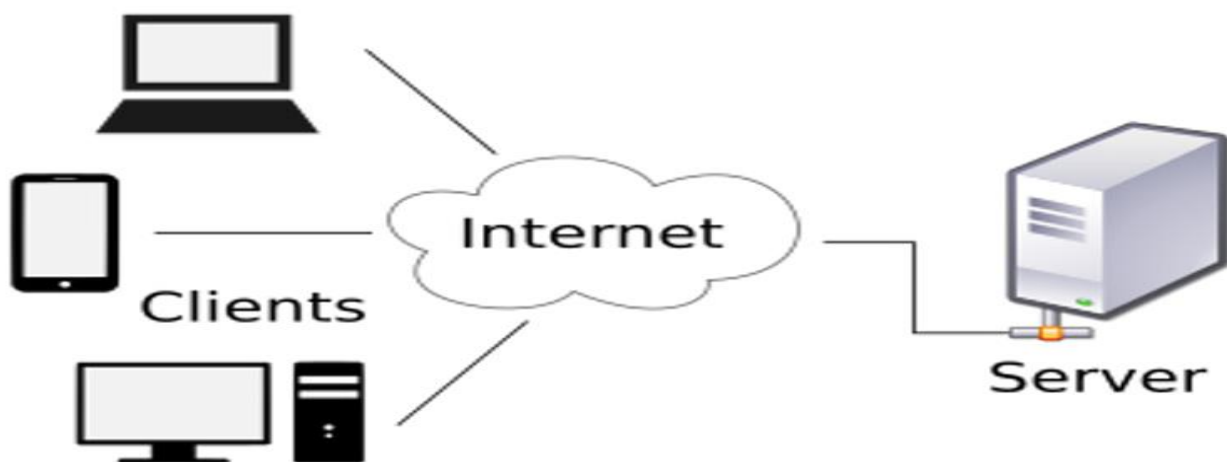
: The Client/Server Model

هو أسلوب لتنظيم العمل داخل الشبكات بحيث يتم توزيع المهام ل :

Server	Client
هو الجهاز او البرنامج الذي يقدم خدمة معينة مثل توفير موقع الكتروني , قاعدة بيانات او ملفات الخادم , يكون جاهزاً دائماً لاستقبال الطلبات	هو البرنامج الذي يطلب خدمة من السيرفر , مثلاً متصفح انترنت يطلب صفحة ويب

ملاحظات :

1. كل Application هو Program , لكن ليس كل Program هو Application .
2. ان ال client هو Application و هو الذي يتخاطب مع ال server اما ال user فهو يستخدم ال Application فقط.
3. ال server يعمل دائماً , تكون ال sockets موضوعه بحالة listening بانتظار ال request من ال client .



ال client هو من يبدأ الاتصال بال server , حيث يكون ال server بانتظار requests من ال client ويقوم ال server بإعادة response حسب الطلبات .

مثلاً : Search engine is server , web browser is client :

: Peer-to-Peer Model (p2p)

هو طريقة اتصال بين الأجهزة بحيث لا يوجد Server كما في النموذج السابق, بل تتوزع المهام بين الأجهزة نفسها , حيث يكون لكل جهاز دور مزدوج :

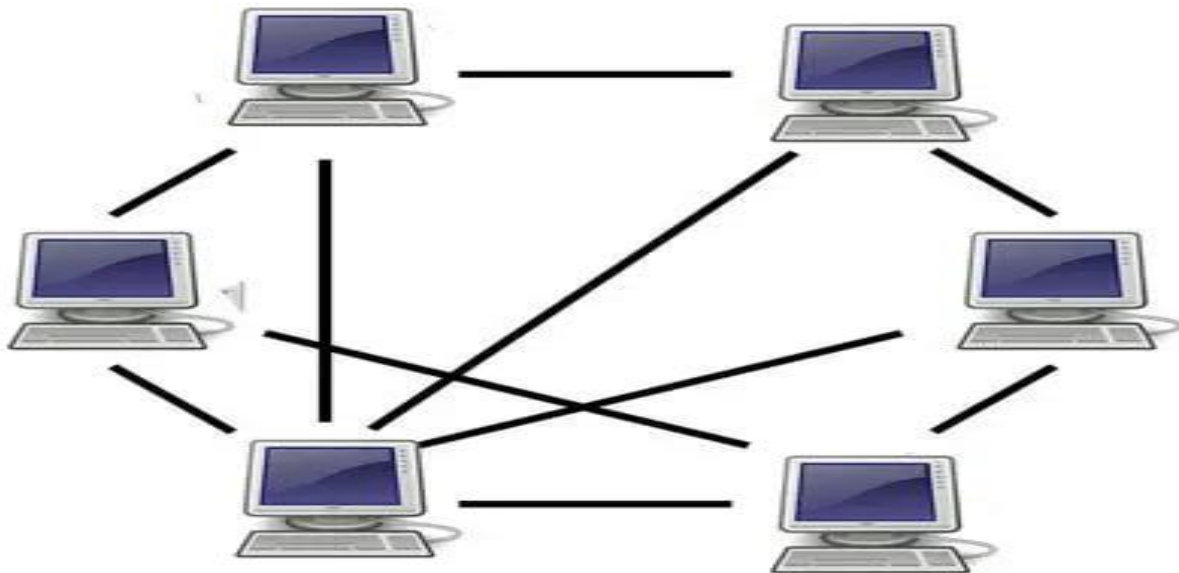
Consumer of resource

Provider

حيث ان :

1. كل جهاز بالشبكة يعتبر (Peer) و كلهم متساويين في الصلاحيات.
2. كل جهاز يمكنه ارسال و استقبال البيانات بنفس الوقت.
3. المهام و الملفات تكون موزعة بين الكثير من الأجهزة و ليس في مكان واحد.

Peer To Peer



مثال :

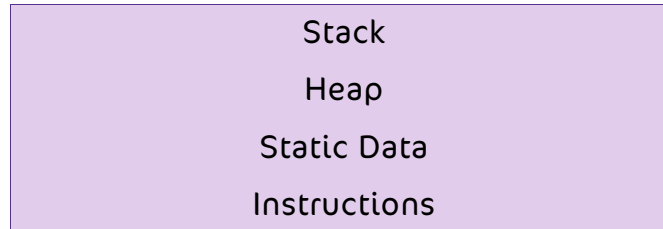
برنامج µTorrent , هو برنامج لمشاركة و تحميل الملفات باستخدام بروتوكول BitTorrent P2P , و هنا الجهاز الذي يحمل الملفات يستقبل جزء من الملف و يرسله الى اجزاء أخرى في نفس الوقت.

Processes

أولاً لتوضيح بعض المفاهيم :

- Program : هو مجموعة تعليمات مكتوبة بلغة برمجة جاهزة للتنفيذ لكنه ليس بالضرورة أن يكون موجه للمستخدم النهائي (user) .

- Application : هو برنامج موجه مباشرة للمستخدم النهائي (user) ويهدف لتقديم خدمة له .
- Process : هو التشغيل الفعلي للبرنامج في الذاكرة مع المعطيات والذاكرة المحجوزة وكل جزء يشغله البرنامج أثناء عمله .
- بنيتها داخل الذاكرة (للتوضيح) :



يمكن لأكثر من Process أن ترتبط بنفس البرنامج .

فمثلاً:

تم فتح 3 نوافذ في المتصفح، كل نافذة عملية process ، حيث أن كل عملية مرتبطة بالمتصفح نفسه لكن منفصلة في الذاكرة.

Inter-process communication (IPC)

يؤمن نظام التشغيل طرقاً لتمكين العمليات من تبادل البيانات ، لأن الـ Processes لا يمكنها مشاركة المتغيرات بشكل مباشر فهي تعمل في مساحات ذاكرة منفصلة.

و ان بعض هذه الطرق:

1. Files : تكتب عملية في ملف وتقرأ عملية أخرى منه .
2. Sockets : يتم الاتصال بين عمليتين على نفس الجهاز أو بين جهازين مختلفين عبر الشبكة حيث تسمح بإرسال و استقبال البيانات .
3. Shared Memory : يتم تشارك جزء من الذاكرة لتبادل البيانات بسرعة كبيرة بين عدة عمليات .

Threads

- هي خطوط تنفيذ مستقلة داخل البرنامج .
- إن الـ Process يمكن ان تحوي على عدة Threads تعمل بشكل متزامن.
- البرنامج يحوي عدة Processes و ان الـ Processes تحوي عدة Threads اي ان الـ Thread مستعمل لاجل وظائف بسيطة .

فمثلاً : المتصفح فيه 3 Threads (عرض الصفحة, الصور, الفيديو...) وكلها تعمل في نفس الوقت بشكل متوازي.

Stack for Thread 1	Stack for Thread 2	Stack for Thread 3
HEAP		
Static Data		
Instructions (Code)		

ال Threads داخل كل عملية تتشارك البيانات الثابتة و الكود البرمجي و ذاكرة ال Heap (للمتحويلات الديناميكية) , و تمتلك بشكل منفصل كلاً من :

1. Stack : فيه المتغيرات المحلية - معاملات التتابع.....

2. كل Thread لديه نسخة خاصة به من مسجلات المعالج : PC - Stack Pointer

3. Thread -Specific Data

Threads vs Processes

	Processes	Thread
Independence	مستقلة عن باقي العمليات	مرتبطه بعملية و هي جزء منها
Address Space	لكل عملية Address Space منفصل	جميع ال Threads داخل عملية ما تشترك بنفس ذاكرة العناوين
Switching	Slow لانها تحتاج الى تبديل السياق	Fast لان معظم المواد مشتركة in the same process
Communication	تحتاج ال IPC مثل الملفات وال Sockets والذاكرة المشتركة	اسهل لانه يمكن استخدام نفس الذاكرة المشتركة

State Information	لا تشترك مع ال Processes الاخرى	تشترك مع كل Thread في نفس ال Process ب الكود - البيانات - ال Heap
-------------------	---------------------------------	---

TCP Sockets - Based Applications

سنحدث عن المواضيع التالية :

Tcp Sockets.3

Sockets.2

Inet Address Class.1

Server Sockets.5

Sockets Class.4

ملاحظة :

سنكتب الكود بلغة Java + Spring Boot Framework (عملي و نظري)

Inet Address:

1. هي فئة موجودة في ال package : java.net

2. تمثل IP على مستوى عالي (High-level representation)

3. تدعم IPv4 و IPv6

4. تستخدمها أغلب فئات الشبكات في جافا مثل :

Socket - Server Socket - URL- Datagram socket - Datagram packet

5. تحتوي كلاً من اسم المضيف و ال IP address

6. لا يوجد constructor أي لا يمكن إنشاء object منه ، وإنما لجلب ال IP نستخدم التابع () getByName و هو

static method أي أنها تنتمي للكلاس نفسه InetAddress حيث نرسل لها host name و ترجع IP Address لهذا الاسم.

مثلاً : InetAddress address = InetAddress.getByName ("www.oreilly.com");

فيحدث التالي :

تفصل الدالة خادم DNS محلي يقوم بالبحث عن عنوان ال IP المرتبط مع الاسم مع العنوان .

Example :

```
import java.net.*;
public class OReillyByName {
    public static void main (String[] args) {
        try {
            InetAddress address = InetAddress.getByName("www.oreilly.com");
            System.out.println(address);
            System.out.println(address.getHostAddress());
        } catch (UnknownHostException ex) {
            System.out.println("Could not find www.oreilly.com");
        }
    }
}
```

هذا الكود سيستدعي الدالة `getByName()` للحصول على كائن يمثل عنوان الموقع المدخل , فيكون ناتج التشغيل :

`www.oreilly.com/208.201.239.36`

يمكن عمل بحث عكسي :

```
InetAddress address = InetAddress.getByName("208.201.239.100");
```

```
System.out.println(address.getHostName());
```

فيكون الخرج : www.oreilly.com أي أنه أعطانا اسم المضيف بعد أن مررنا له ال IP .

ملاحظات :

- `getHostAddress()` يعيد عنوان ال IP بشكل نصي فقط بدون وضع اسم المضيف.
- إذا لم يكن للعنوان اسم مضيف ستعيد الدالة `getHostName()` عنوان ال IP كما هو .

Other functions :

1. `getAllByName()` :

```
InetAddress[] addresses = InetAddress.getAllByName("www.google.com");
```

تعيد مصفوفة تحوي كافة عناوين ال IP المرتبطة بال `HostName` , تفيد عندما يكون للخادم اكثر من IP (مثل المواقع الكبيرة التي تستخدم ال Load Balancing)

2. `getLocalHost()` :

تعيد عنوان الجهاز المحلي الذي يعمل عليه البرنامج.

إذا لم يعرف النظام اسم الجهاز عبر DNS ، سيعيد : `localhost 127.0.0.1`

3. `getByAddress()` :

```
Byte[] ip = {127.0.0.1};
```

```
Inet address = InetAddress.getByAddress(ip);
```

تنشئ `InetAddress` مباشرة باستخدام مصفوفة بايتات تمثل عنوان IP .

لا نحتاج الى DNS .

تستخدم غالباً عندما نريد إنشاء عنوان IP يدوياً .

Sockets:

ال Socket هو المكان أو الواجهة التي يستخدمها البرنامج لإرسال و استقبال البيانات عبر الشبكة , بدونها لا يمكن لأي برنامج إجراء اتصال شبكي .

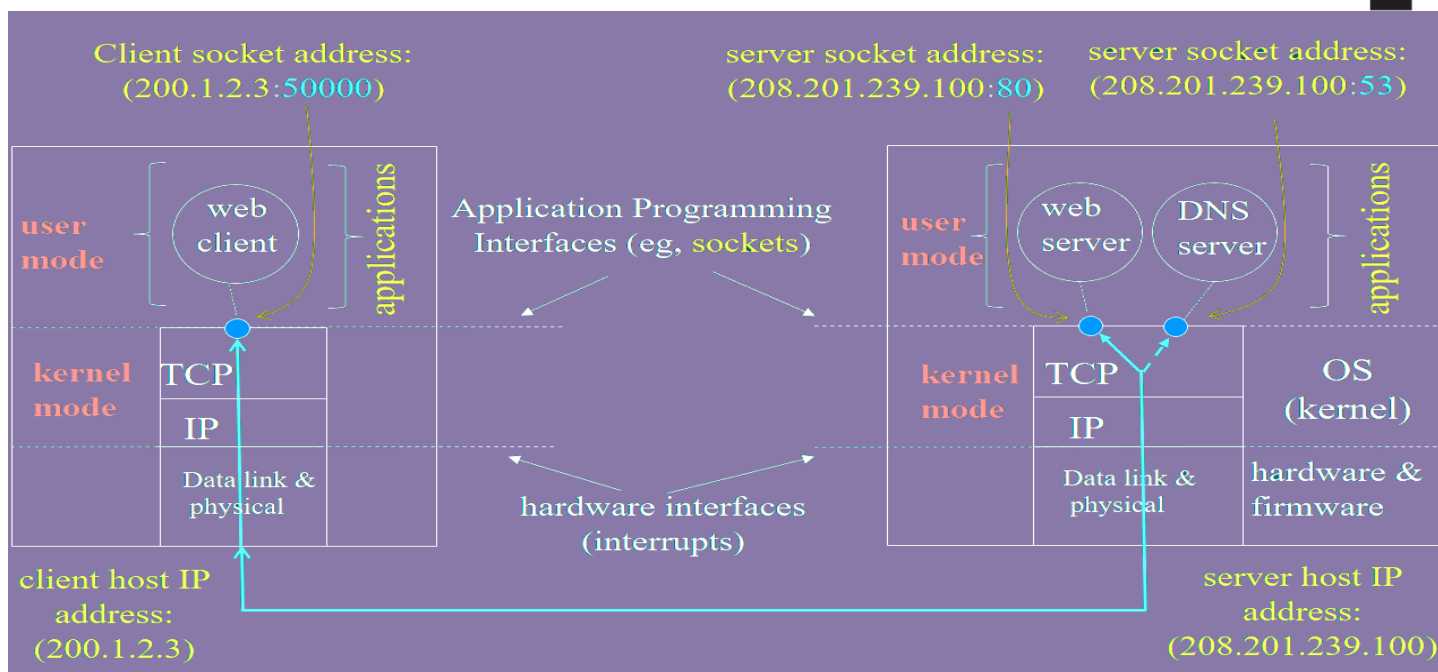
1. يكون تبادل البيانات عبر ال Sockets بين عمليتين أحدهما تمثل ال server و الأخرى تمثل ال Client .

2. يعرف الاتصال باستخدام (2 sockets) 2 endpoints .

3. ال socket هو connection endpoint .

وهو عنوان يكون من : port number , IP address

ex : (208.201.239.36, 80)



تمثل الصورة كيفية تواصل جهاز ال Client مع جهاز ال server.

Socket - Making a Connection :

يريد المتصفح على الجهاز ال Client الاتصال بال Server :

1. ينشئ Socket و يرسل طلب من عنوان السوكيت لديه (200.1.2.3:50000) إلى Socket السيرفر (208.201.239.100:80) .

2. ينتقل الطلب عبر طبقات الشبكة في ال Client و صولاً إلى طبقات الشبكة لل Server .

3. يستقبل نظام التشغيل في ال Server البيانات الواردة و بناءً على ال Port number يوجهها إلى التطبيق المستهدف , هنا هو Web Server .

4. يعالج السيرفر الطلب ويرسل response إلى ال Client Socket (200.1.2.3:50000) في هذا المثال .

Client	Server
Client host IP : 200.1.2.3	Server host IP : 208.201.239.100
200.1.2.3: 50000	208.201.239.100 : 80
هذا يستخدم لتعريف ال Socket وهو IP مع Port number , وقد عرف فيه السوكيت من جهة ال Client .	IP Address و Port Number وبه قد تم تعريف السوكيت من جهة ال Server
: User Mode	: User Mode
تعمل فيه تطبيقات باستخدام و المتاح هو متصفح الويب (في الصورة اعلاه)	تعمل في تطبيقات الخادم مثل الويب و ال DNS

: Kernel Mode الجزء الاساسي من نظام التشغيل يدير الاجهزة و يدير الاتصال بالشبكة و Tcp/IP وتعريف ال data مع ال hardware	: Kernel Mode تدير Stack Tcp/IP و تتعامل مع ال hardware
	: Web Server يكون في حالة ال listening بانتظار الوارد من ال Client على المنفذ رقم 80 : DNS Server يستقبل الوارد على المنفذ رقم 53

Tcp Socket :

إن ال java.net Package يحوي على العديد من ال Classes الخاصة ببناء تطبيقات شبكية مثل :

Tcp Sockets – UDP Socket – URLs – InetAddress.....

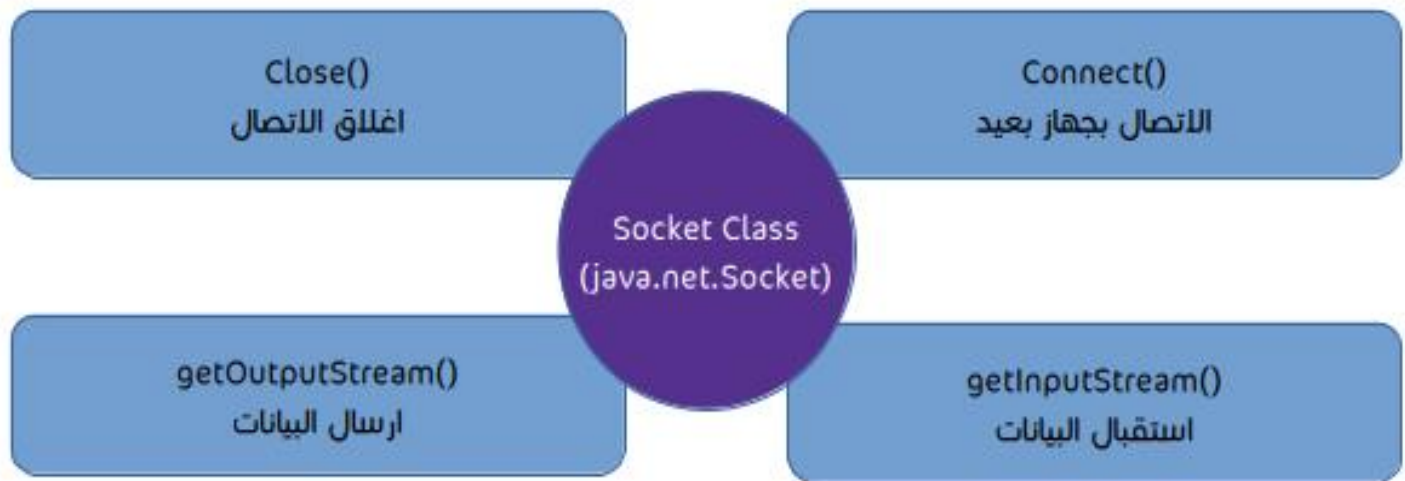
ان ال Sockets هي الأداة الاساسية التي تستخدمها البرامج لإنشاء اتصال عبر الشبكة , فهي تسمح ب :

1. الاتصال بجهاز اخر
2. ارسال بيانات
3. استقبال بيانات
4. إنهاء الاتصال

Tcp Classes in Java :

Socket : يستخدمه ال Client و أحياناً يستخدمه ال Server

Server Socket : يستخدمه ال Server فقط ليستقبل الاتصالات



يمثل برنامج ال Java ال Socket كما يلي :

عندما يكون البرنامج هو ال Client :

1. ينشئ Socket باستخدام ال Constructor مثل :

```
Socket S = new Socket ("hostname",port);
```

يحاول السوكيت إنشاء اتصال بالخادم , واذا كان يعمل فسيحقق الاتصال.

2. بعد نجاح الاتصال يحصل ال Client على Output Stream لارسال البيانات و على ال Input Stream لاستقبال البيانات :

```
InputStream in = s.getInputStream();
```

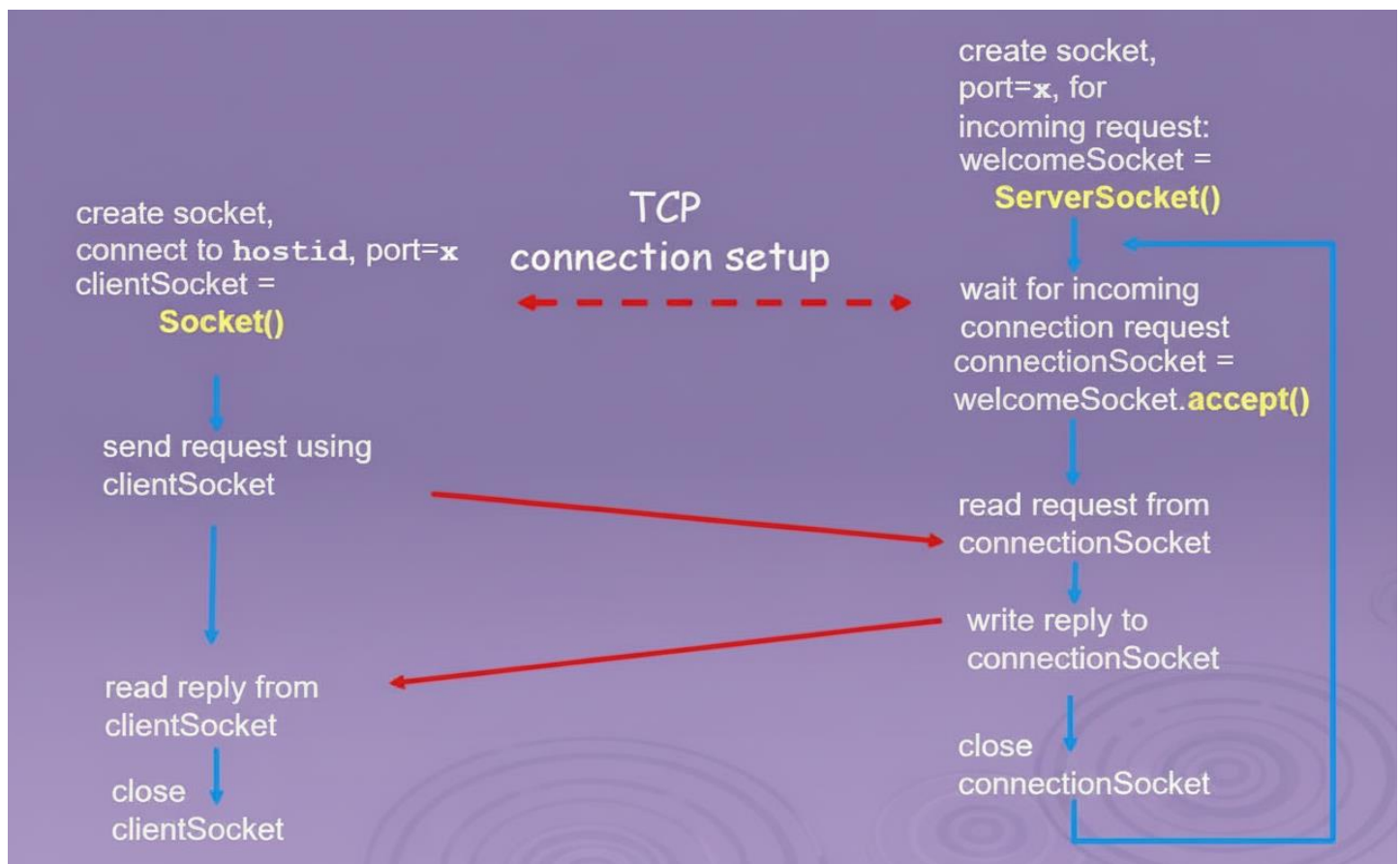
```
OutputStream out = s.getOutputStream();
```

3. ثم يجب إغلاق الاتصال لجعل الموارد المحجوزة متاحة للاتصال اخر:

```
s.close();
```



Tcp Sockets : Client – Server interaction :



1. ينشئ ال Server نقطة اتصال Socket منتظراً Client يتصل به:

```
ServerSocket welcomeSocket = new ServerSocket(port);
```

2. يحاول ال Client الاتصال بالسيفر فينشئ Socket :

```
Socket ClientSocket = new Socket(hostid, port);
```

وهنا يحصل Tcp 3way HandShake (تم ذكرها سابقا في المحاضرة) .

3. قبول اتصال ال Client من قبل ال Server :

```
connectionSocket = WelcomeSocket.accept();
```

ولكل Client جديد يتم إنشاء `connectionSocket` خاصة به ويبقى ال `WelcomeSocket` ينتظر اتصالات أخرى .

4. بعد ان تم قبول الاتصال, يتم انشاء Streams لتبادل البيانات :

```
ClientOut.write();
```

يرسل ال Client بيانات, ثم تنتقل البيانات عبر Tcp الى ال Server , فيستلم الطلب :

```
ClientRequest = ConnectionIn.readline();
```

```
ConnectionOut.write();
```

فيستلم الرد :

```
ServerReply = ClientIn.readLine();
```

5. إنهاء الاتصال من جهتين :

بعد انتهاء المهمة (على هذا الاتصال فقط) لأن Welcome Socket للسيفر ستبقى تعمل :

```
ClientSocket.close();
```

```
connectionSocket.close();
```

Socket Class:

سنكتب برنامج يتصل ب Server خاص بإرجاع الوقت الحالي في ال NIST .

فائدة :

NIST (National Institute of Standards and Technology) هي مؤسسة حكومية أميركية مسؤولة عن تطوير وحفظ المقاييس والمعايير الوطنية للولايات المتحدة ، بما ذلك المعايير المتعلقة بالوقت والتردد، يمتلك سيرفرات معتمدة عالمياً تعيد الوقت والتاريخ الدقيقين وفق التوقيت العالمي UTC ، باستخدام بروتوكولات مثل:

Day Time protocol(port 13)

NTP protocol(port 123)

■ نعرف Socket الاتصال لل Client :

```
Socket mySocket = new Socket("time.nist.gov",13);
```

■ أحد خوادم NIST التي تعيد الوقت وهي تعيد الوقت من عام 1970 حتى اللحظة بالثواني :

```
MySocket.SetSoTimeOut(15000);
```

■ يضبط مهلة القراءة الى 15 ثانية ، اي انه عملية القراءة اذا لم يصل شيء خلال 15 ثانية سوف يعطي exception انه انتهى وقت الاتصال .

```
MySocket.close();
```

■ يتم اغلاق ال Socket و تحرير الموارد.

Writing to the server :

■ يجب أنشاء output stream من ال Socket لإرسال البيانات الى السيرفر:

```
OutputStream out = mySocket.getOutputStream();
```

- الآن يجب كتابة ما سيتم إرساله في ال Stream :

```
byte[] data= {};
```

```
out.write(data);
```

- الآن نريد إرسال الذي تم كتابته , ولكن الذي نريد إرساله هو نص , لذلك نعمل Wrapping باستخدام `OutputStreamWriter` أو `DataOutputStream` و نحدد الترميز الذي نريد للداتا ان ترسل به :

```
Writer mwriter = new OutputStreamWriter(out, "ASCII");
```

```
mwriter.write("This is a message sent to the server");
```

```
mwriter.flush();
```

يمكن أن نضيف Buffer ليعطي أداء أفضل مثلاً إذا استخدمنا `BufferedWriter` يجمع البيانات في Buffer ثم يرسلها دفعة واحدة حيث يقلل ذلك عمليات الإرسال الصغيرة, لكن هنا يجب استخدام flush لضمان النقل.

ملاحظة :

نستخدم `flush()` , عندما نريد إرسال المحتوى مباشرة دون انتظار .
أما `close()` فهي تغلق قناة الاتصال و تنهي الاتصال و يتم تحرير الموارد بعدها .

Reading from the server :

- إنشاء قناة استقبال `InputStream` من `Client` ال Socket :

```
InputStream in = mySocket.getInputStream();
```

- قراءة البايتات المرسله من السيرفر :

```
byte[] data= {};
```

```
in.read(data);
```

يحاول القراءة حتى طول المصفوفة و يعيد عدد البايتات المقروءة حالياً .

- لقراءة البيانات بصيغة نصية لا ننسى ان نغلفها ب `InputStreamReader` أو `BufferedReader` :

```
InputStreamReader reader = new InputStreamReader(in,"ASCII");
```

```
int c = reader.read();
```

تم القراءة محرف محرف لذلك قد تكون بطيئة لذلك غالباً نستخدم `BufferedReader` .

- إذا أردنا قراءة النص كامل نستخدم `StringBuilder` :

```
StringBuilder time = new StringBuilder();
```

```
for (int c = reader.read(); c != -1 ; c = reader.read() ) {
```

```
time.append((char) c); }
```

▪ القراءة باستخدام BufferedReader :

```
BufferedReader readme = new BufferedReader(reader);
```

```
String line = readme.readLine();
```

Example 1 : DayTime

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class DaytimeClient {
    public static void main(String[] args) {

        try (
            // 1
            Socket mysocket = new Socket("time.nist.gov", 13)) {

            //2
            InputStream in = mysocket.getInputStream();

            StringBuilder time = new StringBuilder();

            InputStreamReader reader = new InputStreamReader(in, "ASCII");

            //3
            for (int c = reader.read(); c != -1; c = reader.read()) {
                time.append((char) c);
            }

            //4
            System.out.println(time);

        } catch (IOException ex) {
            System.err.println("Could not connect to server... ");
        }

        finally {
            if (socket != null) {
                try {
                    //5
                    mysocket.close();
                }
            }
        }
    }
}
```

العميل (client) يتصل بسيرفر time.nist.gov على البورت 13, عند نجاح الاتصال يرسل الخادم التاريخ والوقت (Date/Time) كرسالة نصية, ثم يتم غلق الاتصال.

1. إنشاء الاتصال (Connection Setup):

- يتم إنشاء قناة اتصال TCP مع الخادم.
- قد يحدث Blocking (حجب) في هذه المرحلة إلى حين تأكيد نجاح الاتصال أو فشله.

2. تهيئة القراءة (Stream Initialization):

- يتم استخدام InputStream لقراءة البيانات الخام المستلمة من السيرفر.
- يُستخدم StringBuilder لتجميع السلسلة النصية التي تمثل الوقت.
- يتم استخدام InputStreamReader لتحويل تدفق البايتات إلى أحرف باستخدام ترميز ASCII.

3. القراءة والتجميع (Reading and Aggregation):

- تتم قراءة البيانات حرفاً حرفاً من السيرفر.
- تتم إضافة كل حرف إلى الكائن من نوع StringBuilder الذي أطلقنا عليه اسم time.

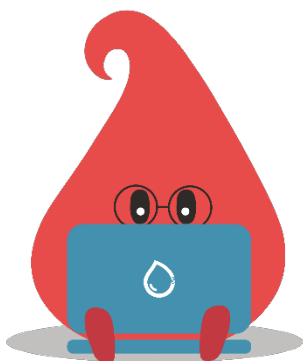
4. إظهار النتيجة (Output):

- طباعة الوقت المستلم على الشاشة.

5. الإغلاق (Closing):

- يتم إغلاق الاتصال بالخادم, وهذا يضمن إغلاق كل القنوات المفتوحة (Streams/Sockets).

وتمت معالجة كل الاستثناءات (Exception Handling) التي يمكن أن تحدث أثناء عملية الاتصال والقراءة (مثل فشل الاتصال أو خطأ في الإدخال/الإخراج).



Example 2 : "Hello world" Client Application

```
import java.io.*;
import java.net.*;

public class Hello{
public static void main(String[] args) {
try {
//1
Socket socket = new Socket("www.EchoHello.org", 50001);
socket.setSoTimeout(15000);
//2
OutputStream out = socket.getOutputStream();
Writer writer = new OutputStreamWriter(out, "UTF-8");
BufferedWriter mwriter = new BufferedWriter(writer);
//3
InputStream in = socket.getInputStream();
InputStreamReader inr = new InputStreamReader(in, "UTF-8");
BufferedReader reader = new BufferedReader(inr);
//4
mwriter.write("Hello World...! \r\n");
mwriter.flush();
//5
String line = reader.readLine();
System.out.println(line);
}
//6
catch (IOException ex) {
System.err.println("error while connecting to the server...");
} finally {
if (socket != null) {
try {
//7
socket.close();
} catch (IOException ex) { // ignore }
}
}
```

يقوم العميل (client) بالاتصال بالخادم (server) لإرسال رسالة "Hello world" واستقبال الرد.

1. إنشاء الاتصال (Connection Setup):

- يتم إنشاء الاتصال بخادم موجود على العنوان www.EchoHello.org.
- يتم الاتصال عبر Port 50001.

2. تهيئة الإرسال (Output Stream Setup):

- إنشاء OutputStream لإرسال الرسالة إلى الخادم.

3. تهيئة الاستقبال (Input Stream Setup):

- إنشاء InputStream للاستقبال الرد من الخادم.

4. إرسال البيانات (Sending Data):

- إرسال البيانات (الرسالة) إلى الخادم.
- تم استخدام الدالة flush لإرسال البيانات بشكل فوري دون انتظار امتلاء المخزن المؤقت (Buffer).

5. قراءة الرد (Reading Response):

- قراءة الرد من الخادم (و هنا تم قراته كسطر واحد).

6. إدارة الأخطاء (Error Handling):

- معالجة الاستثناءات (Exceptions) التي قد تحصل أثناء عمل البرنامج.

7. الإغلاق (Closing):

- إغلاق الاتصال وتحرير الموارد.

ServerSocket Class

ServerSocket Class هو الكلاس المسؤول عن إنشاء سوكيت (Socket) خاص بالخادم، ويوفر كل ما يلزم لاستقبال اتصالات من ال client ببروتوكول TCP.

ماذا يفعل:

إنشاء server باستخدام ال Constructor مثلاً : `ServerSocket server = new ServerSocket(8000);`

يقوم Listening to connections

ضبط الإعدادات قبل بدء الاتصال من دوال/وظائف أخرى مثل:

`toString() , close() , isBound() , isClosed().....`

Server's Basic Life Cycle :**1. إنشاء الخادم باستخدام ServerSocket**

```
serverSocket server = new serverSocket(8000);
```

2. listening for clients

```
Socket clientSocket = server.accept();
```

تعمل هذه الدالة في وضع ال Blocking وترجع كائن Socket يمثل قناة الاتصال .

3. استقبال وإرسال البيانات

من وإلى ال client باستخدام ال Socket's I/O Streams

4. التواصل بين ال client و server

بالبروتوكول المتفق عليه. هنا تتم قراءة الطلبات و معالجه البيانات المستلمة من العميل، و ارسال الردود، و القيام بالعمليات المطلوبة.

5. إغلاق الاتصال (Closing the connection)

قد يقوم بالإغلاق ال client أو ال server أو كلاهما معاً. وهذا ينهي قراءة وكتابة البيانات داخل القنوات التي سيتم اغلاقها.

6. العودة للانتظار

يعود السيرفر للاستماع والانتظار لاتصالات جديدة من عملاء آخرين (يعود للخطوة 2).



Example 3 : DayTime Server

```

import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.util.Date;

public class DaytimeServer {
    public static void main(String[] args) {
        try (
            //1
            ServerSocket server = new ServerSocket(13)) {
            //2

            while (true) {
                try (
                    //3
                    Socket connection = server.accept();
                    //4
                    Writer out = new OutputStreamWriter(
                        connection.getOutputStream(),
                        StandardCharsets.US_ASCII)) {
                        Date now = new Date();
                        //5
                        out.write(now.toString() + "\r\n");
                        //6
                        out.flush();

                    } catch (IOException ex) {
                        // معالجة خطأ متعلق باتصال العميل
                    }
                } catch (IOException ex) {
                    System.err.println("Could not start server: " + ex.getMessage());
                }
            }
        }
    }
}

```

يبين هذا الجزء كيف يتم تهيئة `ServerSocket`، وهنا في هذا المثال على المنفذ 13 (daytime). ثم يتم عمل حلقة انتظار لكل اتصال جديد ومعالجته :

1. ربط المنفذ :

ينشئ الخادم ويربطه بالمنفذ (البورت) المحدد، ويعطي exception اذا فشل الربط ويغلق البرنامج.

2. حلقة الاتصالات :

حلقة ليبقى الخادم يعمل ويستقبل عملاء حتى يتم ايقاقه يدويا او يوقفه exception طارئ .

3. قبول الاتصال (Accepting):

يستخدم الخادم الدالة `accept()` التي تسبب `Block` (حجب) حتى يتم قبول اتصال جديد. عند نجاح هذه الدالة، يعود سوكت (Socket) يمثل القناة مع `client`.

4. إنشاء `Writer` :

ننشئ كائن `Writer` من `OutputStreamWriter` بترميز ما.

5. تكوين الرسالة وإرسالها:

نأخذ الوقت الحالي كـ `String` ونقوم بتشكيل الترميز. نضيف `\n\r` (سطر جديد) كي لا ينتظر `client` سطر جديد في النص.

6. إغلاق القنوات :

نرسل البيانات كلها ونفرغ الـ `Buffer` (باستخدام `flush` لضمان إرسال كل شيء قبل غلق الاتصال).

ملاحظة: يفضل بداية هذا البلوك بالـ `try-with-resources`، والذي يتم فيه إغلاق الاتصال تلقائياً (يشمل كل القنوات والسوكت) حتى لو حدث `exception` أثناء التنفيذ.



Example 3 : "HelloWorld" Server ■

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;

public class HelloServer {
    public static void main(String[] args) {
        int port = 50001;
        try (
            //1
            ServerSocket server = new ServerSocket(port)) {
            System.out.println("Server listening on port " + port);
            //2
            while (true) {
                try (
                    //3
                    Socket connection = server.accept();
                    //4
                    BufferedReader reader = new BufferedReader(
                        new InputStreamReader(connection.getInputStream(),
StandardCharsets.UTF_8));
                    BufferedWriter writer = new BufferedWriter(
                        new
OutputStreamWriter(connection.getOutputStream(), StandardCharsets.UTF_8))
                    ) {
                        //5
                        String line = reader.readLine();

                        //6
                        if (line != null) {
                            writer.write(line);
                            writer.write("\r\n");
                            writer.flush();
                        }
                        //7
                    } catch (IOException ex) {
                        System.err.println("Error handling client: " +
ex.getMessage());
                    }
                } catch (IOException ex) {
                    System.err.println("Could not start server: " + ex.getMessage());
                }
            }
        }
    }
}
```

1. إنشاء مقبس الخادم (ServerSocket Instantiation)

إنشاء ServerSocket و ربطه بال Port : 50001.

2. Listening Loop

حلقة لامتناهية لاستقبال ال clients.

3. قبول الاتصال (Accepting Connection)

قبول اتصال ال client , مما يحجب تنفيذ اي شيء لحين اتصاله.

4. إعداد المخازن المؤقتة (Buffer Setup)

إنشاء Buffers للقراءة من ال client و الكتابة إليه.

5. قراءة البيانات (Reading Input)

قراءة المرسل من ال client.

6. إرسال الاستجابة (Sending Response)

إرسال الرد.

7. إغلاق القنوات :

يتم الاغلاق تلقائيا باستخدام try-with-resources

Closing Server Socket :

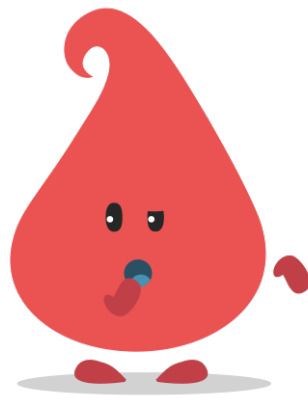
إن إغلاق ال socket أي إنهاء اتصال بين ال server وال client. وهذا الإغلاق لا يؤثر على ال port ولا يمنع ال server من استقبال اتصالات جديدة.

أما إغلاق ال ServerSocket فهذا يعني إغلاق السيرفر نفسه ، و جعل ال port المحجوز قابل للاستخدام من قبل server آخر.

ويمكن ان يغلق by default اذا :

2. Unexpected stop

1. Program dies



مثال على الطريقتين :

```

ServerSocket server = null;
try {
    server = new ServerSocket(port);
    // ... work with the server socket
} finally {
    if (server != null) {
        try {
            server.close();
        } catch (IOException ex) {
            // ignore
        }
    }
}

```

```

ServerSocket server = new ServerSocket();
try {
    SocketAddress address = new InetSocketAddress(port);
    server.bind(address);

    // ... work with the server socket
} finally {
    try {
        server.close();
    } catch (IOException ex) {
        // ignore
    }
}

```

هنا انشأنا ServerSocket دون تعريف البورت مع
ال Constructor ثم انشأنا SocketAddress من خلال
bind InetSocketAddress(port) ثم استخدمنا دالة
لربط ال port بال Socket , وقد يرمي exception اذا
كان ال port مشغول .

ثم تم اغلاق السيرفر في ال finally لضمان تحرير
ال port

انشاء سيرفر مع حجز البورت ضمن الانشاء , ثم اغلقه
باستخدام close في ال finally واي خطأ اثناء
الاغلق يتم تجاهله

ملاحظة:

إن طريقة ال Bind() أكثر أماناً لأن:

1. إنشاء ServerSocket() مع تمرير رقم البورت قد يرمي exception.
2. أما new ServerSocket() دون تعريف رقم ال port فلن يرمي Exception ابداً فيكون ال إنشاء آمناً، و
يمكن أن يحدث أخطاء فقط عند الربط وليس عند الإنشاء.

THE END